MejzlikEdgeCut

Rhino 7 plug-in - user manual

Version 1.3.0

MejzlikEdgeCut generates cutting surfaces for a propeller blade solid in Rhino 7. It replaces the manual workflow of sectioning the blade, drawing polylines at the leading and trailing edges, shifting them until each enclosed polygon has the same area, and lofting the result.

The plug-in produces three surfaces. The **leading edge** and **trailing edge** surfaces are solved so that the cross-sectional area cut off at each station matches a target, which may be constant or vary along the blade. The **spar** surface is placed at a chord fraction measured from the leading edge. All three can overhang the blade so they trim cleanly.

> **Before you start.** This plug-in automates a geometric construction, it does not validate it. The span axis, the leading and trailing edge identification and the station range are all derived from the solid by heuristics that can be wrong on an unusual model. Check the first run of any new blade against geometry you trust before relying on the output.

1. Installation

1.1 What you need

The plug-in is supplied as source code, so it has to be compiled once on a machine that has a build toolchain. After that, the resulting file can be copied to any machine running Rhino 7 and installed there without a compiler.

To compile you need one of:

- The .NET SDK, version 6 or newer, from dotnet.microsoft.com/download
- Visual Studio 2019 or 2022 with the .NET desktop development workload

You also need the .NET Framework 4.8 targeting pack, offered by the Visual Studio installer under Individual Components, and an internet connection for the first build so the RhinoCommon package can be downloaded.

1.2 Compiling

Unzip the project folder anywhere and double-click **build.cmd**. It generates a unique plug-in identifier on the first run, finds whichever build tool you have, compiles, and then opens Windows Explorer with the finished file selected.

```
MejzlikEdgeCut\bin\Release\MejzlikEdgeCut.rhp
```

If you prefer the command line, or want to build in Visual Studio, the folder also contains a solution file:

```
dotnet restore
dotnet build -c Release
```

1.3 If the build fails

The script keeps the window open and lists the likely causes. In order of probability:

Package version. The project pins a RhinoCommon version that may no longer be available. Check nuget.org/packages/RhinoCommon for the newest 7.x release and edit the PackageReference line in MejzlikEdgeCut.csproj. Do not use an 8.x package; it will not load in Rhino 7.

Missing targeting pack. Install the .NET Framework 4.8 targeting pack through the Visual Studio installer.

File in use. Rhino locks the .rhp file while the plug-in is loaded. Close Rhino before rebuilding.

1.4 Installing in Rhino

Drag **MejzlikEdgeCut.rhp** onto an open Rhino 7 window. Alternatively run the **PlugInManager** command, click Install, and select the file.

To confirm it loaded, open PlugInManager and look for Mejzlik Edge Cut. The Description panel shows the version and a summary of what the release does. Then type the command name at the Rhino command line:

```
MejzlikEdgeCut
```

The command prints a banner with its version number every time it runs, so you can always tell which build is loaded.

> **If the version looks wrong.** After reinstalling a newer build, Rhino sometimes keeps showing the previous description text. Restart Rhino. If it persists, uninstall the plug-in in PlugInManager and install the new file.

2. How it works

Understanding the sequence makes the options meaningful, and makes it obvious which one to change when a result looks wrong.

2.1 Finding the blade axis and edges

The plug-in takes the longest axis of the blade's bounding box as the span axis, and slices the solid with planes perpendicular to it at a number of stations across the span. In each slice it finds the chord as the longest distance between two points on the section. Of the two chord ends, the blunter one is taken as the leading edge and the sharper as the trailing edge. Interpolating those points along the blade gives the leading and trailing edge curves.

> **Check this first.** Both steps are heuristics. If the blade is not roughly aligned with a world axis, the stations come out skewed. If the leading and trailing edges are swapped, turn on the SwapLeTe option. Switch on BakeEdgeCurves and look at the derived curves before trusting anything downstream.

2.2 Solving the edge surfaces

At each station the plug-in builds a plane normal to the leading edge curve and slices the blade again. This is the section your manual workflow draws the red polylines in. A straight line cuts off the nose region, and the plug-in searches by bisection for the position where the enclosed area equals the target. The trailing edge is handled the same way against its own curve.

The line that closes the region, the one not lying on the airfoil, becomes the cut line. Lofting the cut lines across all stations produces the cutting surface.

2.3 The spar surface

The spar works differently because a chord fraction is only meaningful in a true airfoil section. It uses the spanwise station planes rather than the edge-normal ones. At each station it measures the chord from leading to trailing edge, places a point at the requested fraction along it, and draws the cut line through that point.

2.4 Radial position

Every distribution is a function of r/R. The RadiusDatum option decides where r is measured from. WorldOrigin, the default, measures along the span axis from the world origin, which is a true radius ratio only if the blade sits in its as-mounted position with the hub axis through the origin. BladeRoot measures from the blade's own root, in which case r/R means fraction of the modelled blade length instead.

The command prints which datum it used and the resulting R before it runs. Read that line. If the blade root sits exactly at the origin under WorldOrigin, it warns you, because that usually means the model is not in its as-mounted position and every distribution is silently shifted.

3. Running the command

Type **MejzlikEdgeCut** and select the blade solid. An option list then appears on the command line. Set what you need, in any order, then press Enter or choose Run.

Every setting is remembered between sessions, so a repeat job on a similar blade usually needs no changes at all. The current distributions and cut direction are printed above the options each time round, so you can read back the configuration before committing to it.

3.1 Which surfaces to make

Option	What it does
`LeadingEdge`	Generate the leading edge cutting surface. On by default.
`TrailingEdge`	Generate the trailing edge cutting surface. On by default.
`Spar`	Generate the spar cutting surface. Off by default.

3.2 Area and position distributions

LeAreaDistribution and TeAreaDistribution set the target cross-sectional area for each edge, independently. SparChordDistribution sets the spar position as a chord fraction from the leading edge. All three offer the same four forms.

Option	What it does
`Constant`	One value at every station.
`Linear`	Straight line in r/R between a root value and a tip value.
`Power`	Root and tip values with an exponent. An exponent of 1 is linear; higher values hold the root figure further outboard.
`Table`	Piecewise linear through control points you type. Values are held flat outside the first and last point.

Table control points are entered as pairs of r/R and value, separated by commas. Areas are in the document's units squared; spar values are fractions between 0 and 1.

```
Area: 0.2:12.0, 0.5:9.5, 0.8:6.0, 1.0:3.5
Spar: 0.2:0.35, 0.6:0.30, 1.0:0.25
```

Decimal separator. Always use a dot as the decimal separator, regardless of your Windows language setting. A comma separates one control point from the next, so a comma used as a decimal mark will be misread.

CopyLeToTe duplicates the leading edge distribution onto the trailing edge, useful when the two start the same and then diverge.

3.3 Cut direction

CutDirection sets which way the cut lines run inside each section plane. The cut is still positioned along the chord in every mode; only the direction the line runs changes.

Option	What it does
`Mixed`	Perpendicular to the chord over the aerodynamic span and parallel to world Z over the root end. The default.
`WorldZ`	Vertical everywhere.
`PerpendicularToChord`	Tilted with the local twist everywhere.
`TransitionRR`	The r/R at which Mixed changes over. Default 0.25.
`WorldZSide`	Which side of the transition is vertical. Inboard by default.
`TransitionBand`	Width in r/R over which the direction rotates. Default 0.10. The rotation is smooth, so the lofted surface has no crease. Set to 0 for a hard switch.

> **Expected behaviour.** Changing the cut direction changes the solved offsets. A vertical cut encloses a different region than a tilted one at the same position, so the solver lands somewhere else to reach the same target area. The areas stay correct; the reported offsets will not match a previous run in another mode.

3.4 Surface extension

Cut lines are lengthened at both ends so the lofted surface stands clear of the blade rather than ending exactly on it, where a trim or split is unreliable.

Option	What it does
`Extension`	Fraction added at each end. Default 0.10, maximum 5.0. Set to 0 for lines that stop at the surface.
`ExtensionBasis`	What that fraction is measured against. LocalChord, the default, gives a clearly visible overhang. CutLineLength uses the cut line itself, which equals local thickness, so 10 percent is only a millimetre or two.

3.5 Stations and range

Option	What it does
`Stations`	Number of spanwise stations. Default 20. More stations give a smoother loft at the cost of computation time.
`RootFraction`	Fraction of span skipped at the root, to clear the hub fillet. Default 0.15.
`TipFraction`	Fraction of span where stations stop. Default 0.97.
`SwapLeTe`	Flip the leading and trailing edge identification if the bluntness heuristic guesses backwards.
`RadiusDatum`	Where r is measured from: WorldOrigin or BladeRoot.

3.6 Check geometry

Option	What it does
`BakeEdgeCurves`	Bake the derived leading and trailing edge curves. Look at these first on any new blade.
`BakeCutLines`	Bake the individual cut lines the surfaces were lofted from.

Option	What it does
`BakeAreaRegions`	Bake the closed nose regions the areas were measured on, as planar surfaces.

4. Output and verification

4.1 Layers

Everything is baked onto named layers, created automatically:

Option	What it does
`CUT_LE_SURFACE`	The leading edge cutting surface.
`CUT_TE_SURFACE`	The trailing edge cutting surface.
`CUT_SPAR_SURFACE`	The spar cutting surface.
`CUT_LE_AREAS`	The nose regions the leading edge areas were measured on.
`CUT_TE_AREAS`	The same for the trailing edge.
`CUT_LINES`	The individual cut lines.
`DERIVED_EDGES`	The leading and trailing edge curves.

4.2 Reading the report

The command prints one line per station. For the edges it gives the radial position, the target area, the area actually achieved and the solved offset from the edge point:

```
--- LE: 18/20 stations solved, linear 12.0 at root -> 4.0 at tip
st 00 r/R=0.150 target=10.800000 got=10.799412 d=3.4180
st 01 r/R=0.194 target=10.448000 got=10.447903 d=3.3517
st 19 r/R=0.970 target=4.240000 SKIPPED: section area 3.9112 < target
```

For the spar it gives the chord fraction, the local chord and the absolute distance from the leading edge:

```
--- SPAR: 20/20 stations solved, x/c from LE = constant 0.3
st 00 r/R=0.150 x/c=0.3000 chord=48.220 x=14.466
```

4.3 Verifying the areas

Switch on BakeAreaRegions, then select any region on CUT_LE_AREAS and run the **Area** command. The result should match that station's reported value. This is the most direct check that the constant-area condition actually held.

The regions are baked exactly as measured, without the extension applied to the cut lines, so the numbers agree.

4.4 Skipped stations

A station is skipped rather than approximated when it cannot be solved, and the reason is printed. The common case is near the tip: the whole section area has fallen below the target area, so no cut can enclose it. The loft then ends short of the tip.

The fix is usually a distribution that tapers toward the tip rather than a constant one. A linear or table law that follows the blade's own thinning removes the problem and is generally what the part wants anyway. Reducing TipFraction is the alternative if the outermost stations are not needed.

4.5 Known limitations

The span axis is the longest bounding box axis, so a blade not aligned to a world axis produces skewed stations. The leading edge is identified by bluntness, which is a heuristic. The area solver assumes the enclosed area grows steadily as the cut moves away from the edge, which holds for a normal airfoil nose but may not for an unusual section. The root fillet and hub boss are not handled; keep RootFraction high enough to stay outboard of them. The loft is a plain loft through the cut lines, not refitted, so rebuild it yourself if you need a cleaner surface.

> **In short.** The plug-in reports what it did at every station and bakes the geometry it measured. Both exist so that a wrong result is visible rather than silent. Read the report.